

ТЗ по проекту «Рефералка в Заявках в банк на РКО по API»

История изменений технического задания

Что было изменено	Кто изменил	Когда

1. Цели проекта и критерии их достижения

Целью проекта является внедрение реферальной системы на Заявки отправляемые в банк на РКО по API (реализуется по [проекту](#)). Что позволит получить возможность:

- Просмотра статистики Тензором (и Банками) по Заявкам
- Возможность начислять и просматривать вознаграждения в сторону Тензора

Критерий достижения цели

1. Реализовано SabyNet приложение SabyBank, в котором
 - Тензор создал реферальную программу "Заявки на РКО", где Банки являются участниками данной программы
 - Тензор и Банки видят статистику поступающих Заявок в рамках реф. программы
 - Менеджер Тензора производит вручную начисления вознаграждений по Заявкам (в том числе через ПМО)
2. Возможность отправки в конкретный Банк Заявки на РКО от Клиента Saby определяется участием Банка в соответствующей реф. программе

2. Потенциальный круг пользователей

Функционал по проекту реализовывается **только для сотрудников Тензора** для мониторинга статистики Заявок от клиентов в банки и проставление вознаграждений.

За месяц и 6 месяцев после внедрения - компания Тензор

Менеджеры со стороны Банков - предположительно продуктом пользоваться не будут! (т.к. все (реализуемые по [проекту](#)) Заявки в Банк уходят по API и менеджеры Банков будут работать в своих учетных системах)

3. Концептуальное описание решения

ТЗ проекта [Заявки в банк на РКО по API](#)

1. Создаем приложение SabyBank на основе SabyNet
 - Реферальная программа создается с под-типом "Заявки на РКО". В реф. программе нет возможности указать "Тему отношений", "Виджет обратной связи", "Ссылку на продукт", "Величину вознаграждения" (Вознаграждение будет начисляться вручную)
 - Возможность вручную создать Заявку отсутствует
 - [Под владельцем] Реализуется возможность добавить Партнер-Банк в виде участника к реф. программе
 - [Под владельцем] аккордеон будет содержать страницу "Офферы" и типовые страницы для SabyNet: "Партнеры", "Настройки"
 - [Под партнером] предлагается сделать заглушку либо вообще не давать доступ, т.к. пользователей (со стороны Банка) на текущий момент не будет
2. Реализуется API метод (будет вызываться строго в акк Тензора) возвращающий Аккаунты Банков участников реф. программ приложения SabyBank, в формате
{ "ID_Реф. программы": { "Тип": <Заявки на РКО|...>, "Участники": [{ИмяБанка1, АккБанка1}, {ИмяБанка2, АккБанка2}] }}

3. Мастер заявок на РКО в реестре расчетных счетов, знания о том какие банки предоставляют услугу "Заявки на РКО" получает через API метод (*вызываемый строго в акк Тензора*) описанный в п2.
4. Реализуется API метод (*будет вызываться строго в акк Тензора*) создания корешка Заявки в реферальной системе, с параметрами: ID Реф. программы, АккБанка, ИдентификаторЗаявки, МетаДанныеЗаявки (дата создания, текущий статус обработки, ...).

По АккБанка, через api соц. сети ищется партнер (@Лицо) в текущей бизнес-группе приложения SabyBank, по @Лицо партнера, находится Карта участника указанной Реф. программы. Создается Корешок. На основе Корешка будет строиться вся статистика Заявок по реф. программам

1. В момент создания Вх. банковской заявки, вызывается метод API создания корешка (*вызываемый строго в акк Тензора*) описанный в п4.
2. Реализуется API метод (*будет вызываться строго в акк Тензора*) обновления корешка Заявки в реферальной системе, с параметрами: ИдентификаторЗаявки, МетаДанныеЗаявки (текущий статус обработки, ...)
3. В момент обновления статуса обработки Вх. банковской заявки (в следствии обращения к API Банков), вызывается метод API обновления корешка (*вызываемый строго в акк Тензора*) описанный в п6.
4. Вся статистика в SabyBank строится на основе созданных Корешков
5. В SabyBank со стороны владельца, доступно ручное назначение вознаграждения (в том числе и через ПМО). Что означает редактирование поля Бонус у записи ВидЦеныДокумент, который и является Корешком Заявки

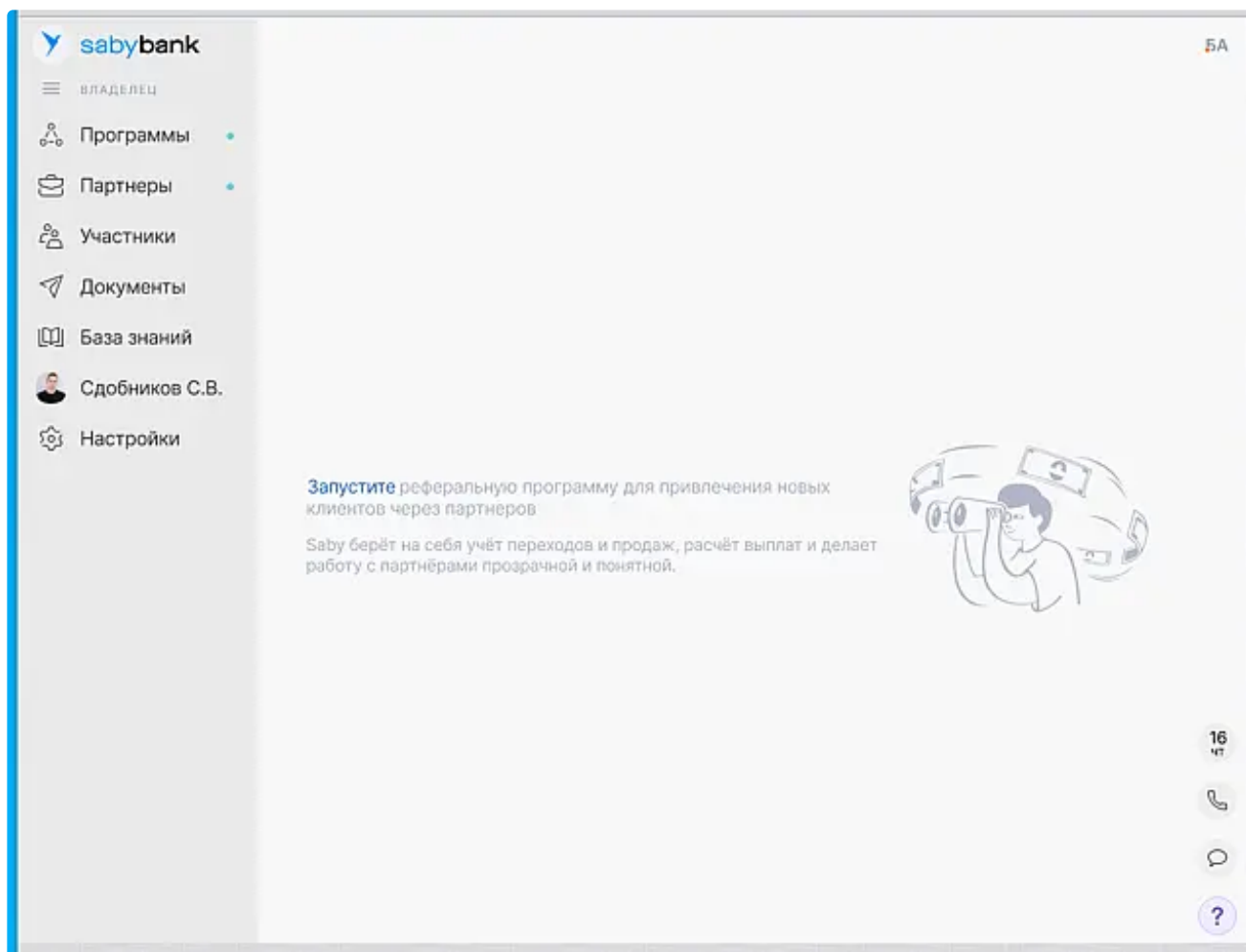
4. Интерфейс

Макет	Состояние макета
SabyBank	Не согласованно

[Бизнес-процесс](#)

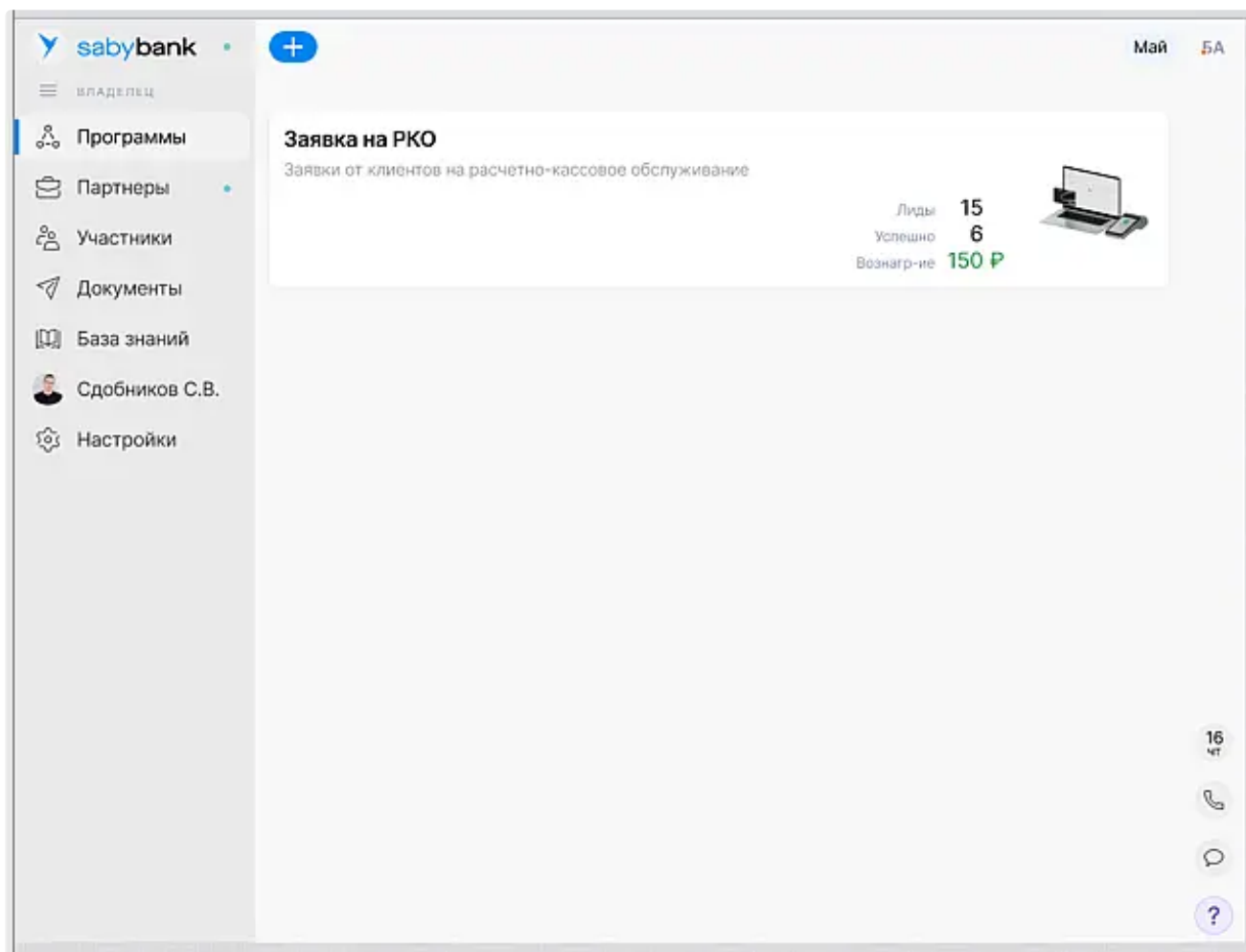
4.1 Содержимое приложения SabyBank

Приложение будет содержать страницу Офферы (Программы) типовые страницы SabyNet: Партнеры, Участники, Настройки.



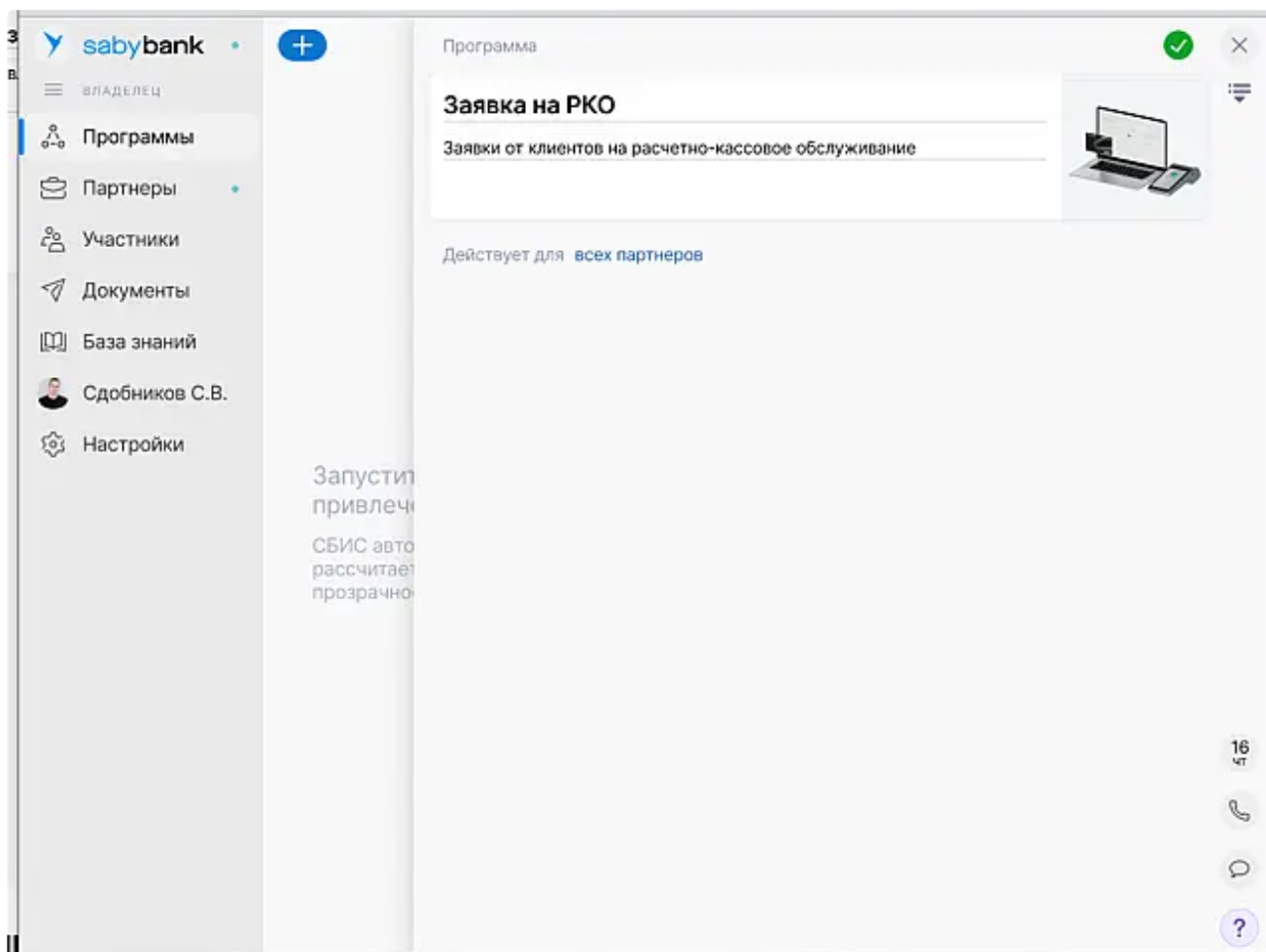
4.2 Страница "Программы"

Страница будет отображаться аналогично странице "Офферы" в реферальной программе, со стандартной кнопкой создания нового оффера, списком офферов в виде плитки, и фильтром по дате для статистики. В плитке будет отображаться название оффера, описание, изображение, и статистика по вознаграждениям.



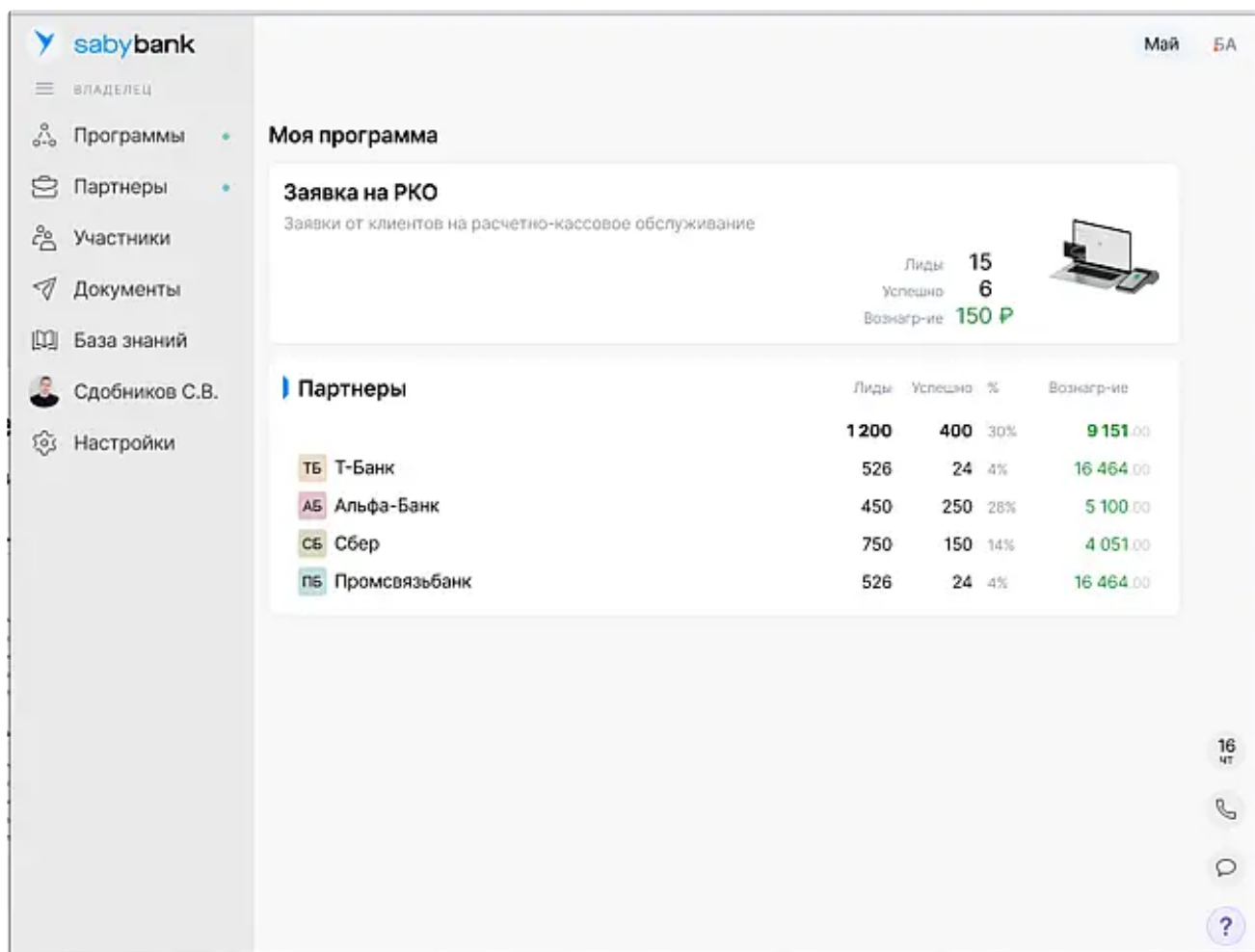
4.3 Карточка создания / редактирования реф программы

В карточке будет доступно редактирование названия, описания, и изображения программы.



4.4 Детализация программы. Страница "Лиды"

При провале в программу на вкладке "Лиды" будет отображаться фильтр по периоду со статистикой вознаграждений, список заявок, и кнопка начисления вознаграждений вручную для конкретной заявки. Доступен фильтр по периоду, статусу и партнеру.



5. Условия лицензирования

В рамках текущего проекта условия лицензирования не применяем

6. Техническое решение

6.1 Архитектура данных и связь понятий

Понятийная модель

▼ Описание понятий

- **Тензор** - Владелец всей схемы, монетизирует отношение Клиента с Банком, встраивая в свои программные продукты функционал для их взаимодействия и вовлечения в него клиентов
- **Клиент Saby** - Пользователь Saby, потребитель банковских услуг, инициатор создания Исх. Банк. Заявок
- **Банк** - Получатель Вх. Банк. Заявок, в рамках которых обрабатывает запросы Клиентов, предоставляя услуги банковского обслуживания. Является Партнером Тензора по реферальной сети
- **Приложение SabyBank** - Приложение на основе SabyNet, содержащее в себе страницы по работе с реферальной системой и формализующее партнерские отношения между Владелцем (Тензором) и его Партнерами (Банками)
- **Банковская услуга (РКО)** - Задается исходящим регламентом "Заявка на открытие счета" и входящим регламентом "Заявка на РКО", а так же экземпляром реферальной программы с типом "Заявка на РКО"

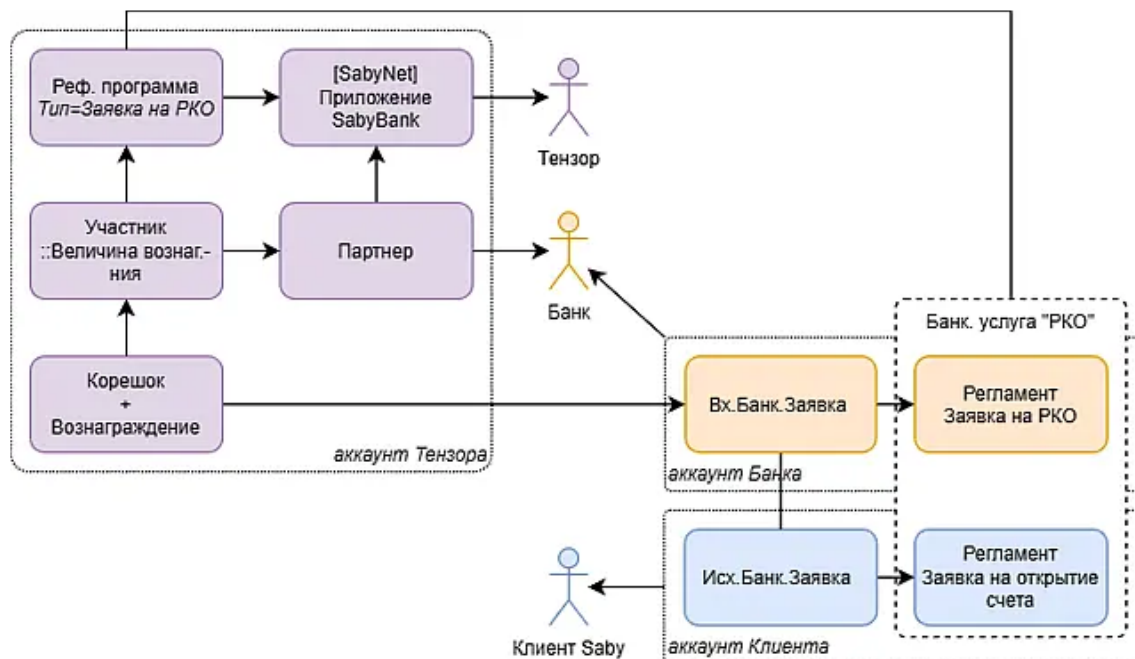
Считаем что **Банк** может предоставлять (для **Клиента Saby**) Банковскую услугу (РКО) только если **Банк**, является Участником Реферальной программы с типом Заявка на РКО

- **Реферальная программа** (с Типом "Заявка на РКО") - Задаёт отражение Банковская услуга (РКО) в реферальной системе. Позволяет задавать Банк-Участника и тарификацию Банка-Участника

Корешок - В аккаунте Тензора (владельца реф. сети) фиксирует факт создания Вх. Заявки. Содержим мета информацию о Заявке (дату создания, текущий статус и т.д.)

- Вознаграждение - Фиксирует величину вознаграждения от Банка в сторону Тензора

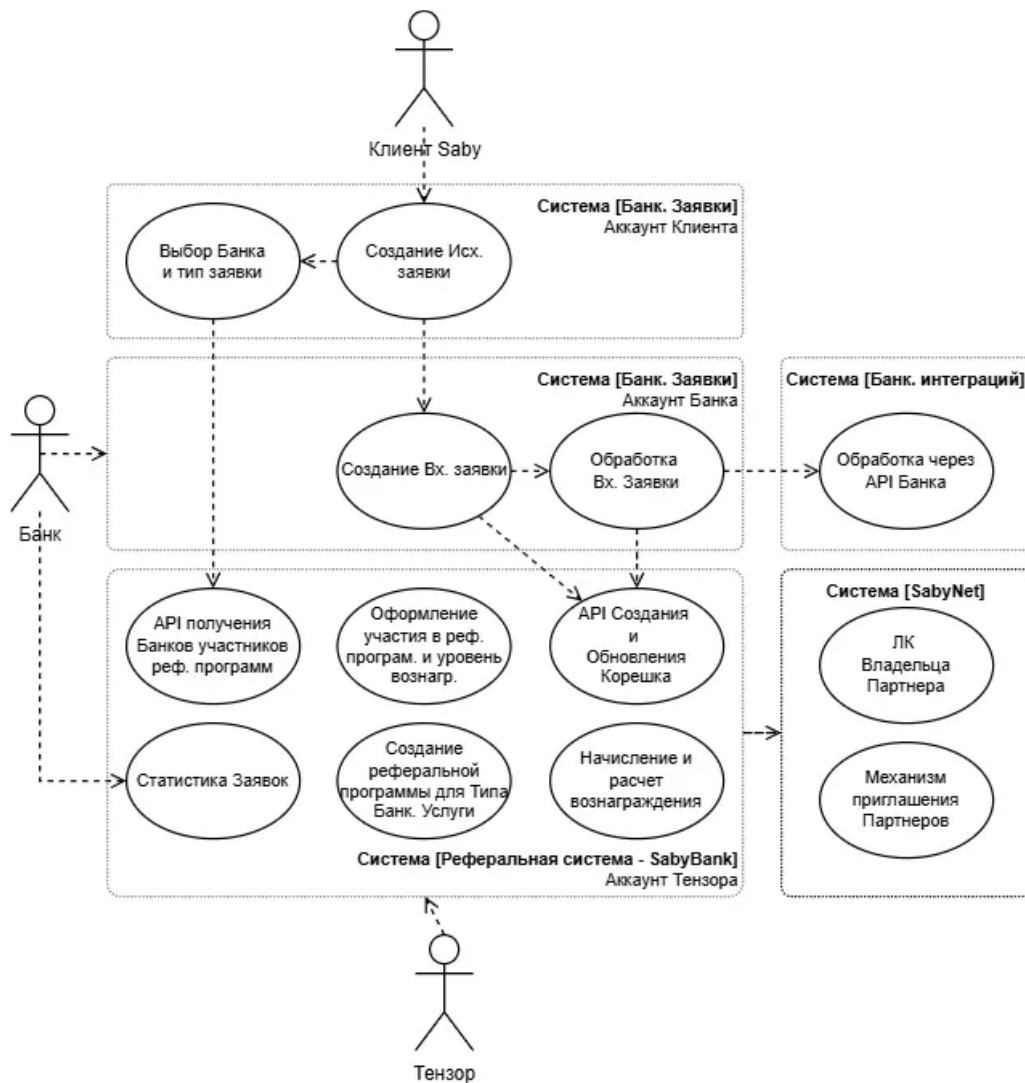
▼ Схема



Подсистемы и зоны ответственности

На схеме обозначены перечень подсистем из которых состоит решение и сценарии за которые каждая из подсистем отвечает и предоставляет

▼ Схема

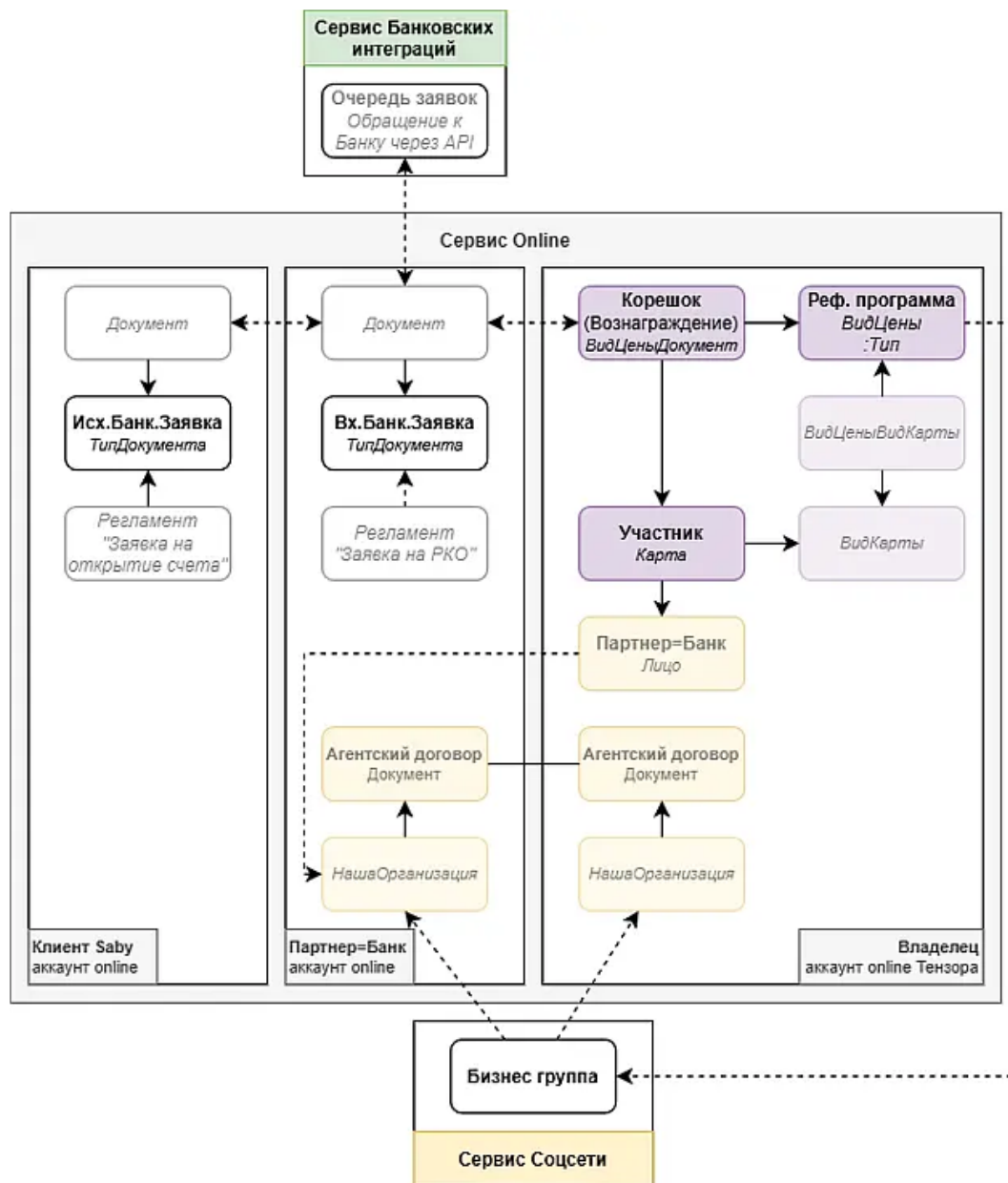


- Точки интеграции Реферальной системы и Банковских заявок сводятся к вызову двух API методов Реферальной системы!
- Каким образом Исх. Заявка попадет в аккаунт Банка в виде Вх. Заявки - Реферальной системе не важно!

Модель данных

[Модель данных лояльности](#)

▼ Схема



Реферальная программа по аналогии с картами и промокодами представляется связкой таблиц [ВидЦены](#) и [ВидКарты](#)

- Название, описание, картинка, иконка, хранится в существующих атрибутах [ВидЦены](#)
 - **Базовая величина вознаграждения за Заявки (Тензору) по реф. программе** хранится в атрибутах [ВидЦены](#)
Решили не автоматизировать сценарии расчета вознаграждения

💡 Для каждого типа Банковской услуги "Открытие РКО", "Кредит" и т.д. будет создан свой подтип Реферальной программы ([ВидЦены.Тип](#))

- **Участник** реферальной программы (Банк-партнер по конкретной реф. программе) представляется записью в таблице [Карта](#) в поле Лицо
 - **Персональная величина вознаграждения за Заявку (Тензору, от конкретного партнера по конкретной реф. программе)** храним в атрибутах [Карты](#) Решили не автоматизировать сценарии расчета вознаграждения
- **Корешок** представляется записью [ВидЦеныДокумент](#), которая связана с **Участником** и **Реф. программой**. Для поиска Корешка вводится новое индексруемое поле [ВидЦеныДокумент.РефЗаявка::UUID](#)
 - **Вознаграждение по Заявке** фиксируется в Корешке в соответствующей записи [ВидЦеныДокумент.Бонусы](#)

Исх. банковская заявка документ в online аккаунте Клиента Saby, Вх. банковская заявка документ (соответствующий Исх. заявке), в online аккаунте Банка = партнера Тензора по реферальной программе

6.1.1 Реф. сеть SabyBank

Создание новой реф. сети

Создадим новую реферальную сеть SabyBank, только для Тензора.

Для этого добавим новый служебный компонент SabyBank ([документация](#) по созданию компонента), только для Тензора (пример - компонент "Биллинг Тензор"). К этому компоненту в дальнейшем будем привязывать новые страницы.

Создадим новую реф. сеть SabyBank (на бою и на тестовых), добавим в нее новый компонент SabyBank.

Все новые страницы (см описание страниц ниже) будем привязывать к этому компоненту, таким образом доступ к новому функционалу будет только у Тензора.

Страница "Офферы"

Для создания программ с типом "Заявка на РКО" создадим новую страницу, текущую страницу "Офферы" в реферальной системе сделок полностью переиспользовать не получится, т.к. при создании программы по кнопке "+" необходимо вызывать метод `ReferralProgram.Create` с дополнительным параметром - тип программы "Заявки на РКО". Страницу привяжем к компоненту SabyBank (см пункт "Реф. сеть SabyBank"), добавим страницу в конфигурацию владельца.

Настройка страницы будет аналогична странице "Офферы" в реферальной системе сделок, за исключением кнопки создания - текущий обработчик `LoyaltyOnline/ReferralLeadNew/Owner/registry:openCreateDialog` будем вызывать с дополнительным параметром: тип программы "Заявка на РКО".

Карточка создания / редактирования реф программы

Для того чтобы скрыть лишние элементы на карточке оффера будут доработаны существующие методы чтения / сохранения реферальной программы `ReferralProgram.Read/Update`, добавим новое поле с типом реферальной программы.

На интерфейсе доработаем существующий компонент `LoyaltyOnline\ReferralLeadNew\Owner\form:View`, если метод чтения вернул тип реф. программы "Заявка на РКО", скроем лишние элементы: выбор сайта, описание вознаграждения, сумма вознаграждения за лид, тему отношений и форму обратной связи.

Страница "Заявки"

Для скрытия лишних элементов доработаем существующую страницу Заявки в системе реферальных сделок. Из метода `ReferralProgram.Read` будем получать тип программы (дорабатывается в п. "Карточка создания / редактирования реф программы"), если это заявка на РКО, скроем лишние элементы (кнопки создания заявки и копирования реф. ссылки).

Отображение заявок и обработка клика будет по этапу "Создание заявок на основе участия в реферальной программе".

Страница "Партнеры"

Для скрытия лишних элементов доработаем существующую страницу Партнеры в системе реферальных сделок. Из метода `ReferralProgram.Read` будем получать тип программы (дорабатывается в п. "Карточка создания / редактирования реф программы"), если это заявка на РКО, скроем лишние элементы (кнопки создания заявки и копирования реф. ссылки), а так же отключим возможность задавать индивидуальные вознаграждения

Виджет "Мои программы"

Создадим новый виджет для владельца и партнера рефсети, в виджете в виде плитки у владельца отобразим все его программы, у партнера - все программы, в которых он участвует.

Для загрузки данных используем существующий метод `ReferralProgram.GetList`, для отображения элемента плитки - компонент `LoyaltyOnline/ReferralLeadNew/tile:Container`

При клике на элемент плитки, в стековой панели покажем подробную информацию о программе, аналогично проваливанию в программу из реестра Офферы. Для этого используем платформенный опенер страниц `SabyPage/pageOpener:Opener`. В параметры передадим идентификатор страницы заявок, у владельца `owner-deals`, у партнера - `participant-deals`.

В панель будет передаваться фильтр по периоду, выбранный на главной странице.

6.1.2 Создание заявок на основе участия в реферальной программе

Создание заявки в зависимости от участия банков в рефпрограммах

Перечень банков, доступных для отправки заявки на РКО, определяется участием Банка в реферальной программе типа «Заявки на РКО» в приложении SabyBank.

Мастер заявок получает перечень банков через API метод (см. [6.3.2. Новые API\Метод "Получить банки-участники реферальных программ SabyBank"](#)), вызываемый строго в аккаунте Тензора.

Если Банк не является участником соответствующей реферальной программы, отправка заявки в такой Банк невозможна.

Создание заявки в аккаунте банка

Остается прежним, как и в [предыдущем проекте](#).

Плюс добавляется вызов асинхронного API метода создания Корешка в реферальной системе (см. [6.3.2 Новые API\Метод "Создать корешок заявки"](#)).

Изменение статуса заявки

При изменении статуса обработки банковской заявки (в результате взаимодействия с API Банка) вызывается API метод обновления Корешка (см. [6.3.2 Новые API\Метод "Обновить корешок заявки"](#)).

Корешок является единственным источником данных для построения статистики в приложении SabyBank.

Реестр "Заявки" на данных из "корешков" в SabyBank

Так как данные для заявок будут вычитываться из "корешков", т.е. полностью изменится бизнес логика и формат возвращаемых значений, создадим новые методы БЛ:

- Списочный метод получения данных о заявках из "Корешков" (см описание данных в п. 6.4), с фильтром по дате, статусу заявки и партнеру.
- Метод получения данных об одной заявке из "Корешков" (см описание данных в п. 6.4)

На интерфейсе реализуем новые компоненты:

- Элемент списка, на основе данных из "корешка"
- Карточку просмотра информации о заявке, будет отображаться более подробная информация - банк, ид. заявки, дата создания, статус.

Назначение вознаграждений

На странице "Заявки", в действии над записью, и по кнопке ПМО, реализуем возможность назначать вознаграждения вручную.

Доработаем существующий метод назначения вознаграждений `ReferralProgram.SetLeadPrice`. В зависимости от типа программы, если это заявка на РКО, метод будет менять данные корешка (ВидЦеныДокумент.Бонусы).

На интерфейсе используем существующий компонент

`LoyaltyOnline/ReferralLeadNew/Owner/_deals/View/ChangePriceDialog`

6.1.3 Создание Корешков Реферальных программ по существующим Заявкам

Так как проект реферальной системы выходит позже проекта отправки заявок через API, требуется обработка ранее отправленных заявок.

В рамках проекта выполняется пакетная обработка уже существующих заявок и создание по ним Корешков в реферальной системе.

Создание Корешков по существующим заявкам выполняется сервисно на основании данных о ранее отправленных заявках.

Повторное создание Корешка по заявке с тем же идентификатором не допускается.

▼ Тех. Детали

Для обеспечения полноты статистики при запуске проекта: по заявкам, уже отправленным в банк до выхода проекта реферальной системы, необходимо создать корешки задним числом.

Контекст

Проект реферальной системы выходит позже проекта отправки заявок по API.

Механика

- Выполняется обработка уже отправленных заявок и пакетное создание корешков в реферальной системе.
- Для создания корешков используется метод "Создать корешок заявки" (вызов из ВНР).

Правила

- Дубли не допускаются (повторное создание по тому же ИдентификаторЗаявки приводит к ошибке дублирования, как и в основном сценарии).

6.1.4 Технический долг & доработки

Выделить работу с реферальной системой в отдельный СБИС модуль

Текущий backend работы с реферальной системой ([орх](#), [реализация](#)) переместить в новый СБИС модуль PriceFormation.Referral (ответственный Тимошенко А.) Поставить в зависимость от него модуля существующий модуль [PriceFormation.Online](#).

Аналогичную работу провести на уровне [юнит-тестов](#).

В реферальных программах сделок перейти на "корешки" и отказаться от получения статистики по лидам (сделкам) в сервисе маркетинга

Для упрощения поддержки и последующих улучшений реферальной системы сделок, переведем хранение данных о заявках на механизм "корешков".

Для того чтобы не дублировать данные о вознаграждениях, во всех местах получения вознаграждений для реферальной системы сделок, добавим дополнительную проверку - будем проверять, что запись в ВидЦеныДокумент не "корешок".

Проверять можно по наличию поля ВидЦеныДокумент.ТипСвязи - будет не пустое только в "корешках".

При создании заявки, будем вызывать метод создания "корешка" (см. описание в п. 6.3.2), в сценариях:

1. При создании заявки из личного кабинета: из существующих методов создания заявки ReferralProgram.CreateLeadForPartner (из под владельца) и ReferralProgram.CreateLeadByPartner (из под партнера)
2. Форма обратной связи - при событии создания сделки. Если источник проставляется асинхронно, то на момент получения события мы не сможем понять что сделка реферальная, и корешок не создадим, будет необходима доработка api создания сделки.

При изменении источника у сделки, если новый источник - реферальный, так же будем создавать "корешок". Для этого необходимо событие смены источника у сделки.

При изменении статуса заявки - вызывать метод обновления "корешка" (см. описание в п. 6.3.2).

Добавим запись в историю реферальной программы, при создании заявки, и при смене источника у сделки на реферальный.

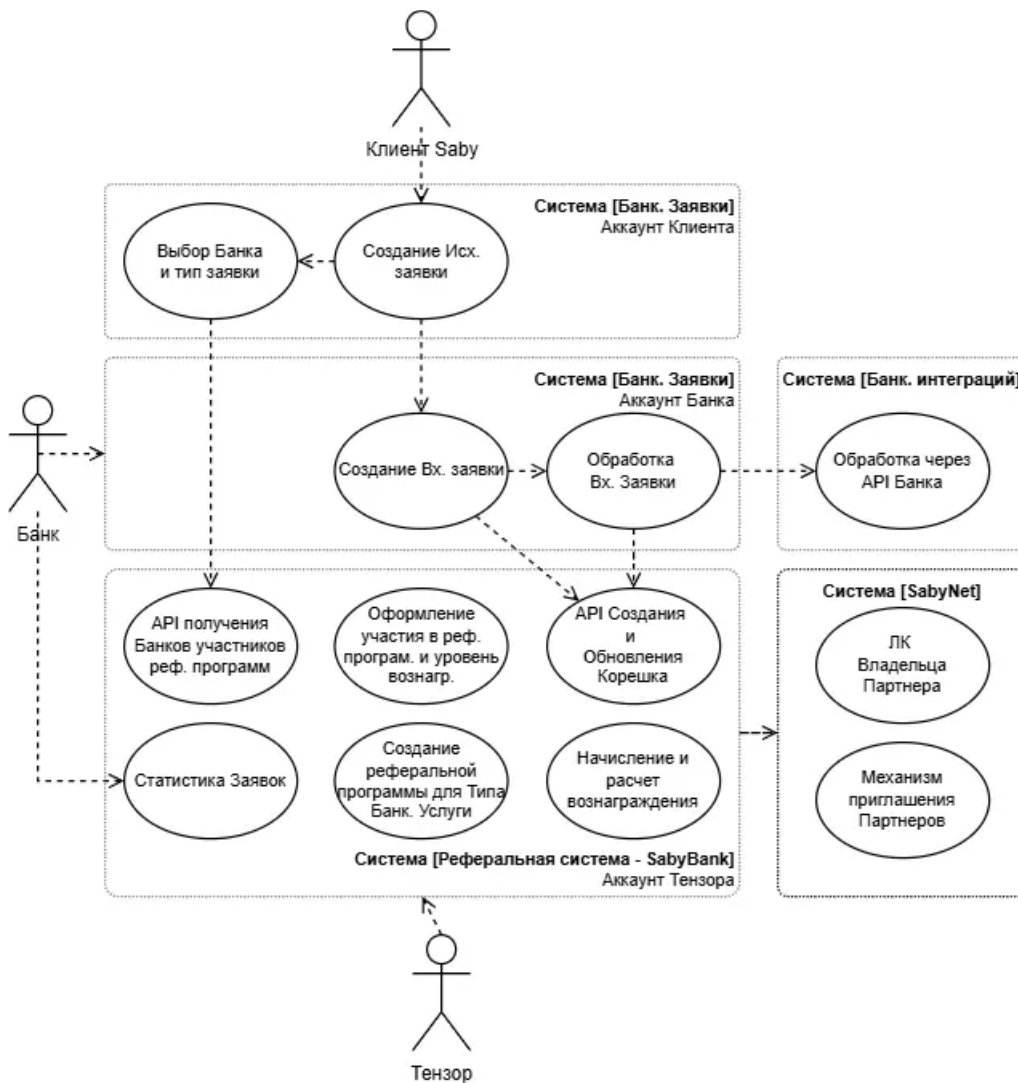
Затем, с помощью ВНР, создадим "корешки" для всех существующих заявок.

После ВНР, во всех местах получения вознаграждений, перейдем на получение данных из "корешков":

- Реестр "Офферы"
- Реестр "Заявки"
- Реестр "Партнеры"
- Список периодов с вознаграждениями (отображается на всех страницах детализации оффера)
- Виджеты реферальных программ

На интерфейсе, для страницы "Заявки", полностью перейдем на метод получения данных из "корешков" (метод и внедрение будет реализовано по этапу "Создание заявок на основе участия в реферальной программе").

6.2 Схема взаимодействия сервисов



- Точки интеграции Реферальной системы и Банковских заявок сводятся к вызову двух API методов Реферальной системы!
- Каким образом Иск. Заявка попадет в аккаунт Банка в виде Вх. Заявки - Реферальной системе не важно!

6.3 Описание API

6.3.1 Используемые API

Использование API других подсистем, сервисов не планируется

6.3.2 Новые API

Метод "Получить банки-участники реферальных программ SabyBank"

Для мастера заявок на РКО: получить список банков, в которые допустимо отправлять заявку, исходя из участия банков в реферальных программах приложения SabyBank.

Контекст вызова

Вызывается строго в аккаунте Тензора.

Результат

Возвращает реферальные программы приложения SabyBank и их участников в формате:

```
{"ID_РефПрограммы": {"Тип": "<Заявки на РКО|...>", "Участники": [{"ИмяБанка": "...", "АккБанка": "...}]}}
```

Примечания

- Метод является единственным источником истины для UI мастера заявок о доступных банках.
- Если банк отсутствует в участниках программы типа "Заявки на РКО", отправка заявки в него недоступна.

Метод "Создать корешок заявки"

Для сценария создания входящей банковской заявки: зафиксировать в реферальной системе "корешок" (служебную запись), на основании которого строится вся статистика и начисляются вознаграждения.

Контекст вызова

Вызывается строго в аккаунте Тензора.

Входные параметры

- ID_РефПрограммы
- АккБанка
- ИдентификаторЗаявки
- МетаДанныеЗаявки:
 - дата создания
 - текущий статус обработки
 - прочие служебные данные (при необходимости)

Алгоритм

- По АккБанка через API соцсети ищется партнер (Лицо) в текущей бизнес-группе приложения SabyBank.
- По Лицо партнера находится Карта участника указанной реферальной программы (реф. код).
- Создается Корешок заявки (запись ВидЦеныДокумент), в котором фиксируются:
 - ссылка на реферальную программу
 - ссылка на участника (карту)
 - идентификатор заявки
 - метаданные заявки

Ограничения и правила

- Повторный вызов метода с тем же ИдентификаторЗаявки не допускается: генерируется исключение (ошибка дублирования).
- Сценарий смены банка по одной заявке маловероятен, но поддерживается как:
 - удаление старого корешка
 - создание нового корешка

Метод "Обновить корешок заявки"

Для сценария изменения статуса обработки входящей банковской заявки: обновить метаданные корешка, чтобы статистика в SabyBank соответствовала фактическому состоянию обработки заявки.

Контекст вызова

Вызывается строго в аккаунте Тензора.

Входные параметры

- ИдентификаторЗаявки
- МетаДанныеЗаявки:
 - текущий статус обработки
 - прочие изменяемые атрибуты (при необходимости)

Алгоритм

- По ИдентификаторЗаявки находится Корешок (ВидЦеныДокумент).
- Обновляются метаданные.
- Изменения сохраняются.

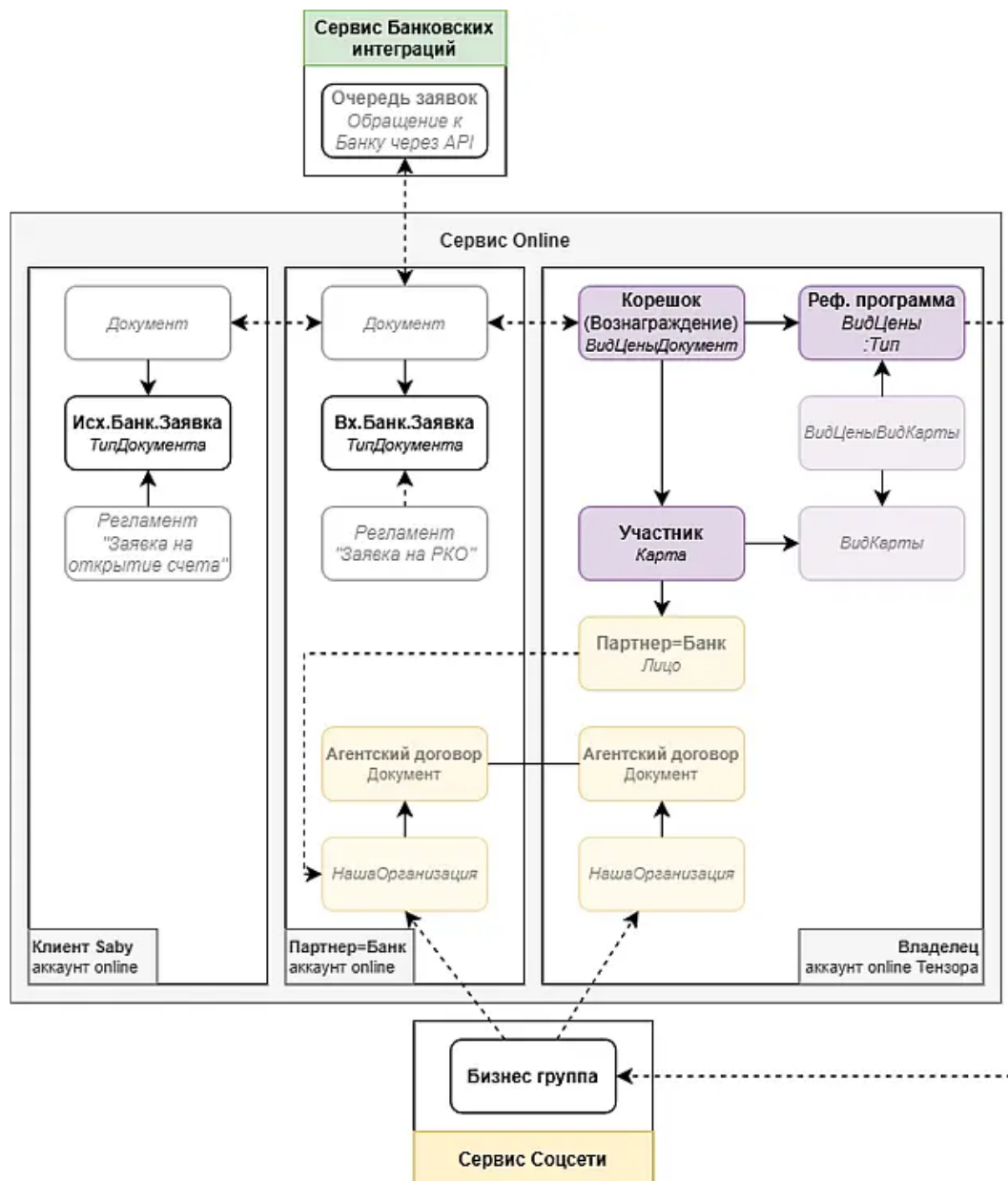
Примечания по производительности и БД

- Объемы: тысячи, в пределах десятки тысяч заявок (ожидаемая нагрузка).
- Требуется проработка индексации/уникальности поиска корешка по ИдентификаторЗаявки (проектируется отдельно).

6.4 Схема хранения данных

Модель данных

[Модель данных лояльности](#)



Реферальная программа по аналогии с картами и промокодами представляется связкой таблиц [ВидЦены](#) и [ВидКарты](#)

- Название, описание, картинка, иконка, хранится в существующих атрибутах [ВидЦены](#)
 - **Базовая величина вознаграждения** за Заявки (Тензору) по реф. программе хранится в атрибутах [ВидЦены](#)
- Решили не автоматизировать сценарии расчета вознаграждения

💡 Для каждого типа Банковской услуги "Открытие РКО", "Кредит" и т.д. будет создан свой подтип Реферальной программы ([ВидЦены.Тип](#))

- **Участник** реферальной программы (Банк-партнер по конкретной реф. программе) представляется записью в таблице [Карта](#) в поле Лицо
 - **Персональная величина вознаграждения** за Заявку (Тензору, от конкретного партнера по конкретной реф. программе) храним в атрибутах [Карты](#) Решили не автоматизировать сценарии расчета вознаграждения
- **Корешок** представляется записью [ВидЦеныДокумент](#), которая связана с **Участником** и **Реф. программой**
 - **Вознаграждение по Заявке** фиксируется в Корешке в соответствующей записи [ВидЦеныДокумент](#). Бонусы

Исх. банковская заявка документ в online аккаунте Клиента Saby, **Вх. банковская заявка** документ (соответствующий Исх. заявке), в online аккаунте Банка = партнера Тензора по реферальной программе

Схема хранения Корешка документа Заявки

Информация о Корешке документа Заявки хранится в таблице [ВидЦеныДокумент](#)

ВидЦены - Идентификатор реферальной программы

Карта - Карта участника реферальной программы

ДатаВремя - Дата и время создания Заявки

OperationID - UUID документа Заявки

ТипСвязи - 10 - Заявка в работе, 11 - Заявка успешно завершена, 12 - Заявка не успешно завершена

EffectiveDate - Дата и время завершения (успешно или не успешно) Заявки

Бонусы - Велечина начисленных вознаграждений (в случае успешно завершения Заявки) иначе null

Атрибуты - Мета-данные о документе Заявки (Аккаунт Клиента Исх. Заявки, Идентификатор Исх. Заявки, прочие данные которые помогут идентифицировать Исх. и Вх. Заявку). по этим данным ни какой поиск не производится.

Поля ВидЦеныДокумент	Заявка создана	Заявка на обработке	Заявка завершена	Заявка с вознаграждением
ВидЦены	Идентификатор реферальной программы			
Карта	Идентификатор карты участника реферальной программы			
ДатаВремя	Дата и время создания Заявки			
OperationID	UUID документа Заявки в аккаунте Партнера			
ТипСвязи	10	10	11-Успех НеУспех	12- 11-Успех
EffectiveDate	NULL	NULL	Дата и время завершения Заявки	
Бонусы	NULL	NULL	NULL	Кол-во вознаграждения
Атрибуты	Мета-данные о документе Заявки			

Новые индексы

Будут созданы новые индексы. См. детали [6.4.1 Описание основных выборов](#)

6.4.1 Описание основных выборов

- Узнать факт участия Партнера в реферальной программе

▼ Детали

- Через api соц.сети в текущей бизнес-группе SabyBank ищем @Лицо Контрагента партнера
- По @Лицо, находим связанные [Карты](#), и оставляем только ту которая связана с [ВидЦены](#) (через таблицу [ВидЦеныВидКарты](#), [ВидКарты](#)), которая является реферальной программой. Таким образом мы нашли Карту и установили факт участия Партнера в конкретной реф. программе

Используется существующий индекс таблицы Карта по (Лицо)

Выборка идентична сценарию "Получение факта участия покупателя во внешней системе лояльности (UDS, PremiumBonus)"

- Получить список заявок по реферальной программе по всем партнерам

▼ Детали

- Отбираем записи таблицы [ВидЦеныДокумент](#), связанные с [ВидЦены](#) (реф. программой), с сортировкой по дате создания "корешка" заявки ([ВидЦеныДокумент](#).ДатаВремя).

Используется существующий индекс таблицы ВидЦеныДокумент по (ВидЦены+ДатаВремя)

Выборка идентична сценарию "Получение списка продаж по скидке\бонусной_программе"

- Для фильтрации по статусам сделки (В работе|Завершена Успешна|Завершена не Успешна) необходимо использовать в запросе поле ВидЦеныДокумент.ТипСвязи. По которому индекса нет. Поэтому будет создан новый индекс на таблицу ВидЦеныДокумент (ВидЦены + ДатаВремя + ТипСвязи)

- Получить список заявок партнера по реферальной программе

▼ Детали

- Через API соцсети по UUID бизнес-группы и идентификатору партнера получаем @Лицо контрагента

2. По @Лицо, находим связанные [Карты](#), и оставляем только ту которая связана с [ВидЦены](#), которая является реферальной программой. Таким образом мы нашли Карту участника конкретной реф. программы
3. Отбираем записи [ВидЦеныДокумент](#), связанные с Картой участника реф. программы, с сортировкой по дате создания "корешка" заявки ([ВидЦеныДокумент.ДатаВремя](#)).

Используется существующий индекс таблицы ВидЦеныДокумент по (Карта+ДатаВремя)

Выборка идентична сценарию "Получение списка продаж по карте лояльности клиента"

- Обновить корешок "заявки"

▼ Детали

1. По UUID заявки ищем (и обновляем) в таблице [ВидЦеныДокумент](#) запись по полю ВидЦеныДокумент.OperationID
- Используется существующий индекс таблицы ВидЦеныДокумент по OperationID

- Получить сумму вознаграждений за период по реферальной программе

▼ Детали

1. Отбираем записи таблицы [ВидЦеныДокумент](#) в диапазоне дат (поле ДатаВремя), по которым есть начисления (Бонусы>0)
2. Подсчитываем сумму полю Бонусы

Используется существующий индекс таблицы ВидЦеныДокумент по (ВидЦены + EffectiveDate + Бонусы <> 0). Но! EffectiveDate подразумевает хранения даты завершения заявки, а мы хотим получить выборку заявок по дате создания. Возможно потребуется новый индекс!

- Получить сумму вознаграждений за период по партнеру

▼ Детали

1. Через API соцсети по UUID бизнес-группы и идентификатору партнера получаем @Лицо контрагента
2. По @Лицо, находим связанные [Карты](#), и оставляем только ту которая связана с [ВидЦены](#), которая является реферальной программой. Таким образом мы нашли Карту участника конкретной реф. программы
3. Отбираем записи [ВидЦеныДокумент](#), в диапазоне дат (поле ДатаВремя), связанные с Картой участника реф. программы
4. Подсчитываем сумму по полю Бонусы

Используется существующий индекс таблицы ВидЦеныДокумент по (Карта + ДатаВремя)

Выборка идентична сценарию "Получение бонусного баланса\бонусных начислений по карте лояльности"

6.5 Пользовательский интерфейс

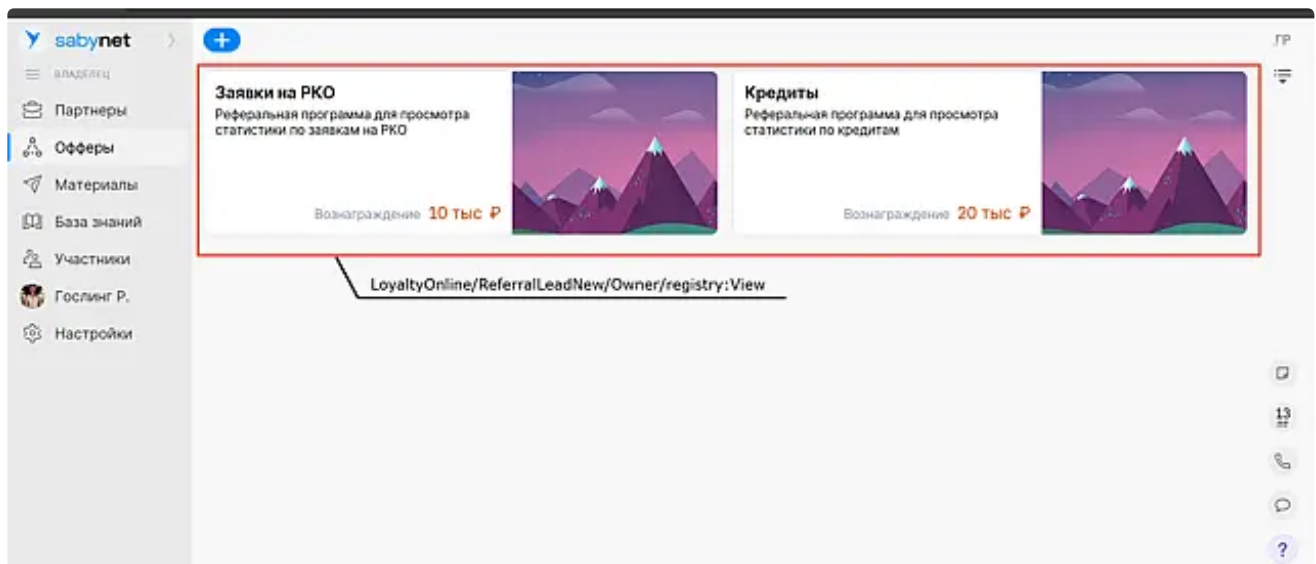
6.5.1 Декомпозиция пользовательского интерфейса

Для реализации интерфейса по проекту в основном будут использоваться и дорабатываться уже существующие компоненты, реализованные для реферальной системы сделок.

Реестр "Офферы"

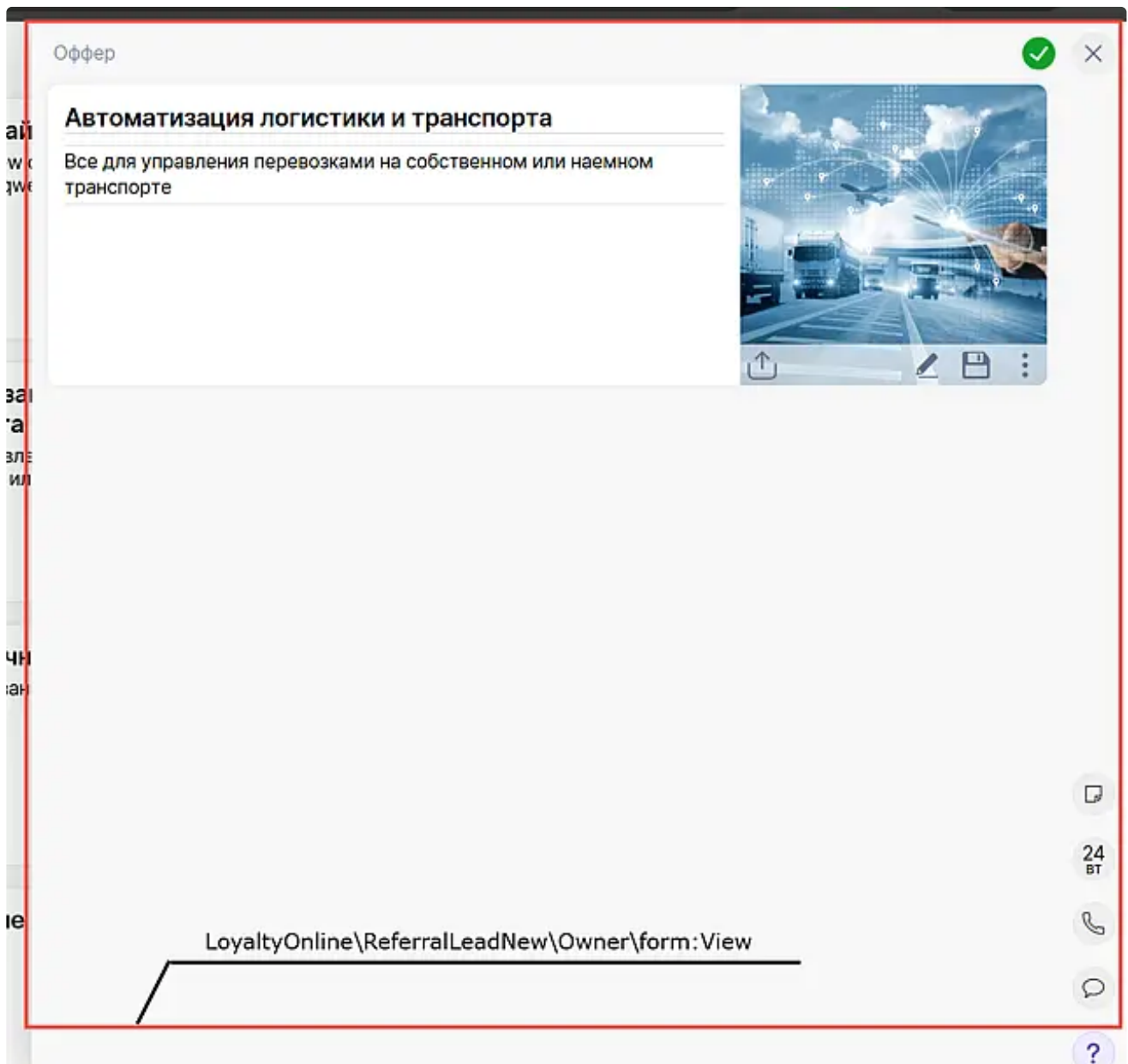
Для отображения реестра используем существующий компонент

[LoyaltyOnline/ReferralLeadNew/Owner/registry:View](#)



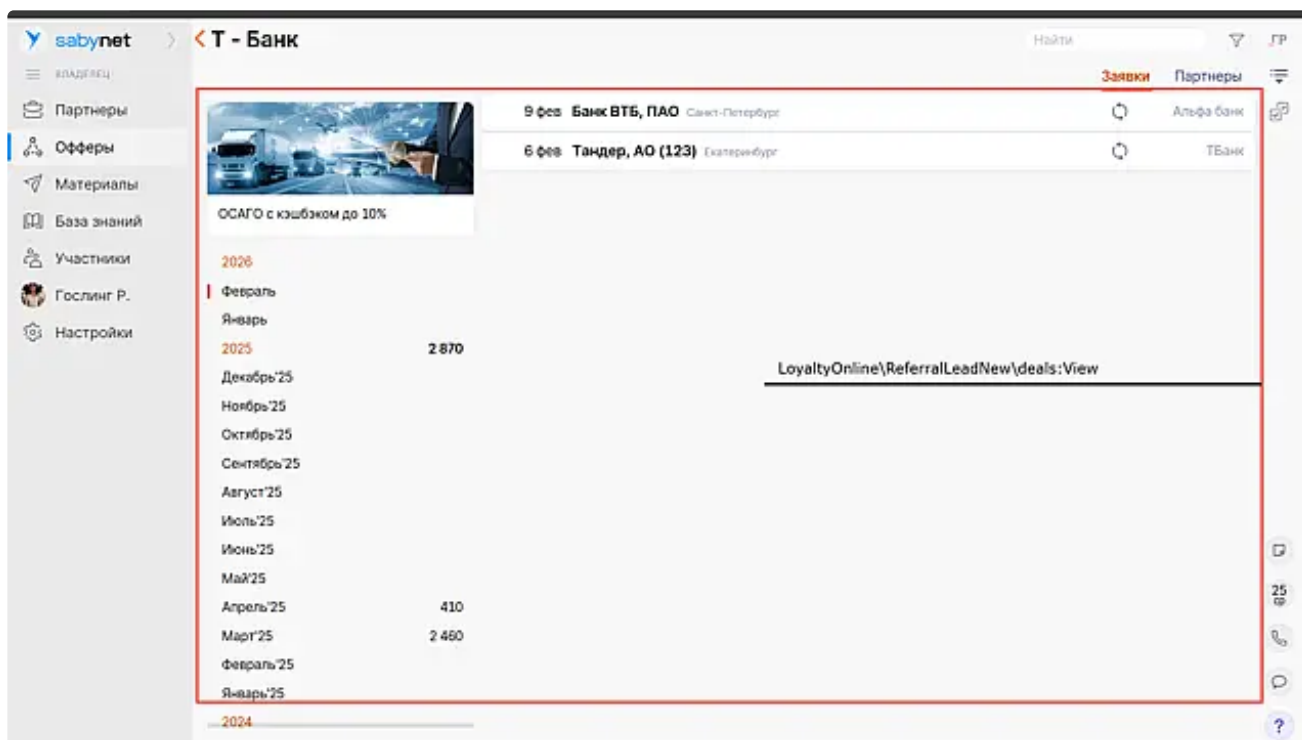
Карточка создания / редактирования реф программы

Используем компонент `LoyaltyOnline\ReferralLeadNew\Owner\form:View`, скроем лишние элементы настройки в зависимости от типа программы.



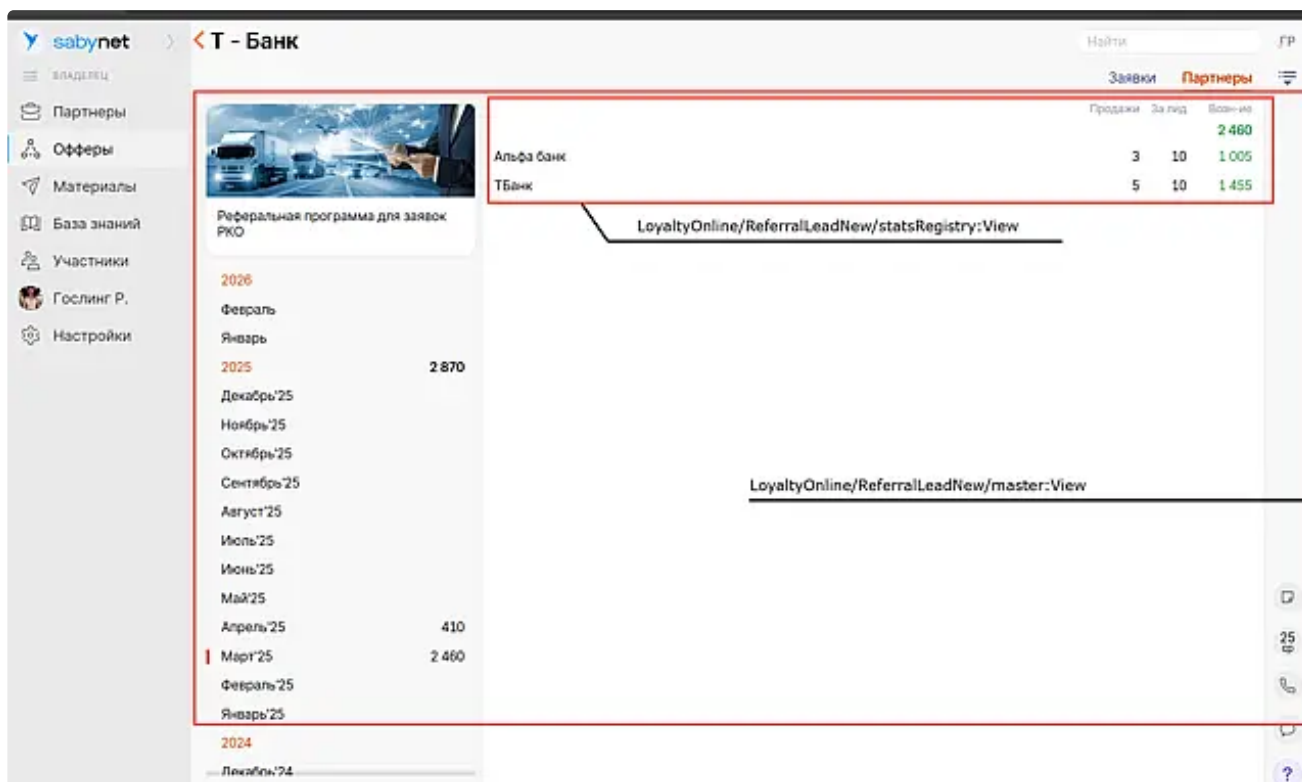
Страница "Заявки"

Используем компонент `LoyaltyOnline\ReferralLeadNew\deals:View`, скроем лишние элементы (кнопки создания заявки и копирования реф. ссылки).



Страница "Партнеры"

Используем компоненты `LoyaltyOnline/ReferralLeadNew/master:View` (компонент на основе платформенного мастер-детейл, с готовой мастер-областью), и `LoyaltyOnline/ReferralLeadNew/statsRegistry:View` (компонент реестра статистики по рефпрограмме, с настраиваемыми колонками). В мастер-области скроем лишние элементы (кнопки создания заявки и копирования реф. ссылки)



6.5.2 Описание источников данных в интерфейсе

Страница "Офферы"

Выполняется на сервисе представления

Методы выполняются параллельно:

PriceFormation.GetPersonBGRoleInAgents - метод получения роли текущей персоны в бизнес-группе, используется для ограничения функционала определенным ролям (редактирование доступно только администраторам и модераторам).

ReferralProgram.GetList/4 - списочный метод получения данных для реестра офферов.

Карточка создания / редактирования реф программы

Выполняется на клиенте

Методы выполняются параллельно:

ReferralProgram.Read/3 - метод чтения информации рефпрограммы.

PriceFormation.GetPersonBGRoleInAgents - метод получения роли текущей персоны в бизнес - группе, используется для ограничения функционала определенным ролям (редактирование доступно только администраторам и модераторам).

Страница "Заявки"

Выполняется на сервисе представления

Методы выполняются параллельно:

ReferralProgram.Read/3 - метод чтения информации рефпрограммы (используется название, описание, тип).

ReferralProgram.GetLeadPeriodList/4 - метод загрузки данных для фильтра по периоду со статистикой.

1-я цепочка методов (выполняется последовательно)

PriceFormation.GetPersonBGRoleInAgents - метод получения роли текущей персоны в бизнес - группе, используется для ограничения функционала определенным ролям (редактирование доступно только администраторам и модераторам).

ReferralProgram.GetLeadList/4 - списочный метод загрузки данных для реестра заявок.

Страница "Партнеры"

Выполняется на сервисе представления

Методы выполняются параллельно:

ReferralProgram.Read/3 - метод чтения информации рефпрограммы (используется название, описание, тип).

ReferralProgram.GetLeadPeriodList/4 - метод загрузки данных для фильтра по периоду со статистикой.

1-я цепочка методов (выполняется последовательно)

PriceFormation.GetPersonBGRoleInAgents - метод получения роли текущей персоны в бизнес - группе, используется для ограничения функционала определенным ролям (редактирование доступно только администраторам и модераторам).

ReferralProgram.GetPartnerList/4 - списочный метод загрузки данных для реестра партнеров.

6.5.3 Реализация прикладных решений

Прикладные решения не планируются.

6.5.4 Доработки интерфейсного фреймворка

Не требуется

6.6 Замеры скорости работы

В параллель идут работы по согласованию внешнего вида реестров и страниц с директором. До окончания этих работ, проводить замеры скорости работы отделом "Тестирование производительности" не целесообразно.

Если к окончанию проекта, данные вопросы по согласованию будут решены, замеры скорости работы будут произведены.

6.7 Хранилище GIT/SVN, модули.

Система лояльности:

<https://git.sbis.ru/warehouse/price-formation/>

Деньги:

<https://git.sbis.ru/buh/money>